

1.4.4 Throughput in Computer Networks

In addition to delay and packet loss, another critical performance measure in computer networks is end-to-end throughput.

To gain further insight into the important concept of throughput, let's consider a few examples.

Figure 1.19(a) shows two end systems, a server and a client, connected by two communication links and a router.

Consider the throughput for a file transfer from the server to the client.

Let R_s denote the rate of the link between the server and the router; and R_c denote the rate of the link between the router and the client.

Suppose that the only bits being sent in the entire network are those from the server to the client.

We now ask, in this ideal scenario, what is the server-to-client throughput? To answer this question,

Clearly, the server cannot pump bits through its link at a rate faster than R_s bps; and the router cannot forward bits at a rate faster than R_c bps.

If $R_s < R_c$, then the bits pumped by the server will “flow” right through the router and arrive at the client at a rate of R_s bps, giving a throughput of R_s bps.

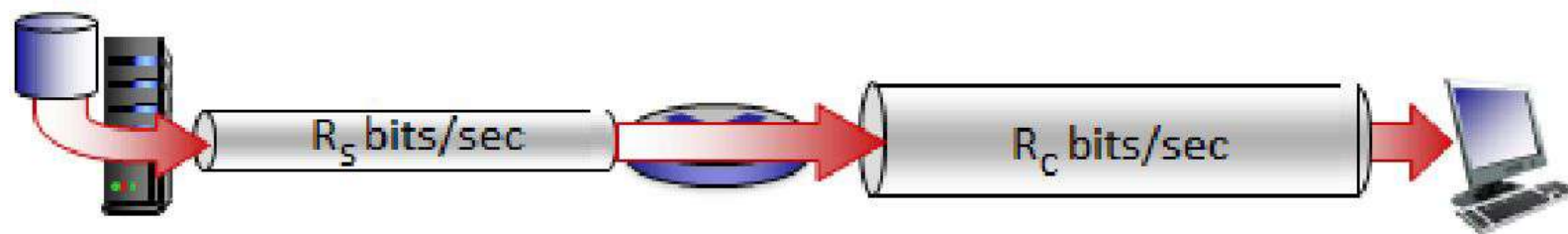
If, on the other hand, $R_c < R_s$, then the router will not be able to forward bits as quickly as it receives them.

In this case, bits will only leave the router at rate R_c , giving an end-to-end throughput of R_c . (Note also that if bits continue to arrive at the router at rate R_s , and continue to leave the router at R_c , the backlog of bits at the router waiting for transmission to the client will grow and grow—a most undesirable situation!)

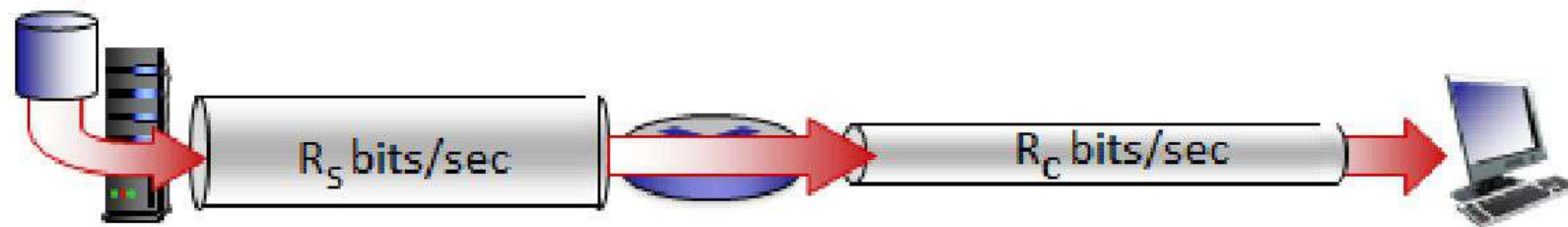
Thus, for this simple two-link network, the throughput is $\min\{R_c, R_s\}$, that is, it is the transmission rate of the **bottleneck link**.

Having determined the throughput, we can now approximate the time it takes to transfer a large file of F bits from server to client as $F/\min\{R_s, R_c\}$.

$R_s < R_c$ What is average end-end throughput?



$R_s > R_c$ What is average end-end throughput?



bottleneck link

link on end-end path that constrains end-end throughput

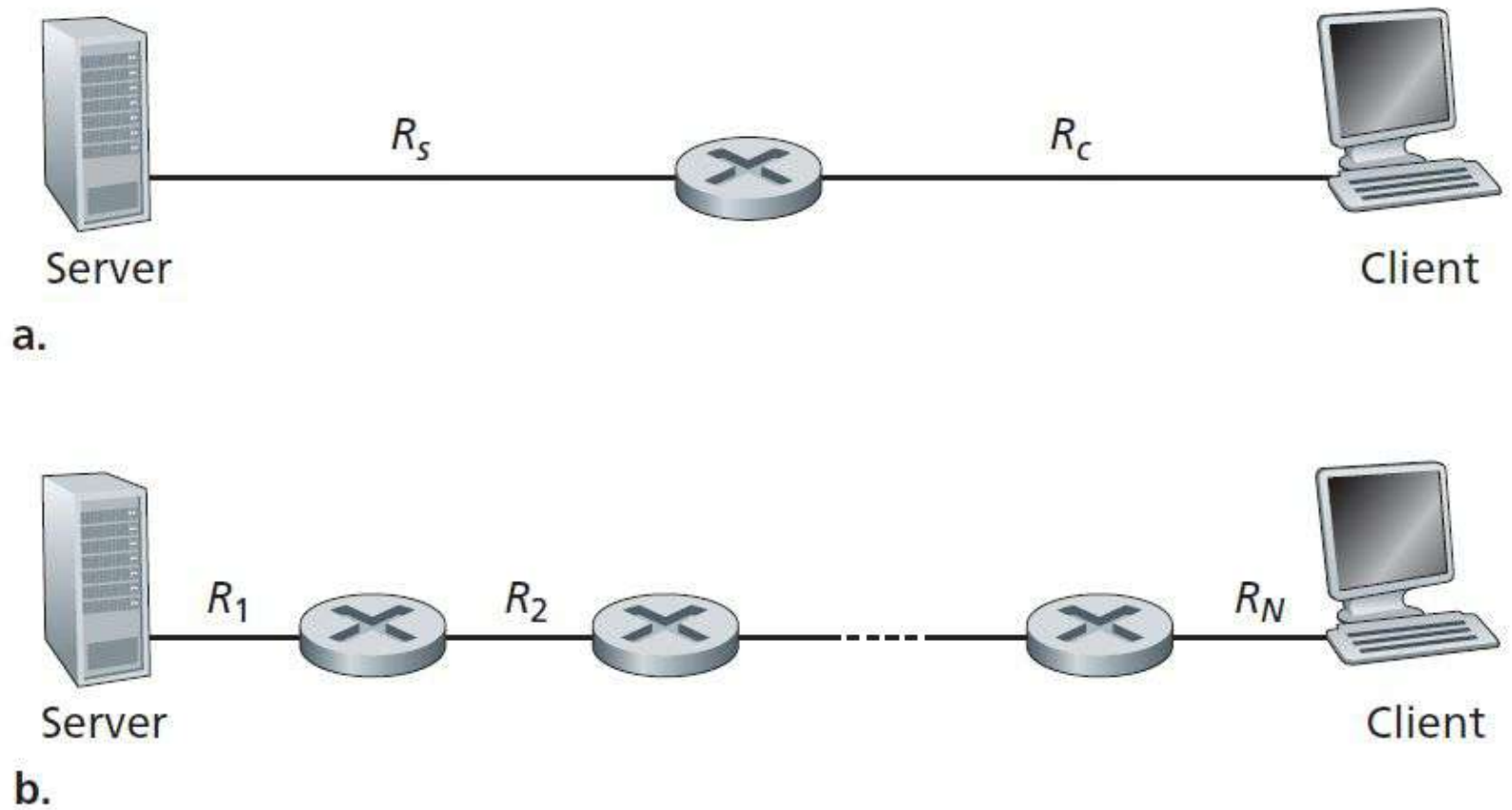


Figure 1.19 ♦ Throughput for a file transfer from server to client

Figure 1.19(b) now shows a network with N links between the server and the client, with the transmission rates of the N links being $R_1, R_2, \dots, R_N$.

Applying the same analysis as for the two-link network, we find that the throughput for a file transfer from server to client is $\min\{R_1, R_2, \dots, R_N\}$, which is once again the transmission rate of the bottleneck link along the path between server and client.

Now consider another example motivated by today's Internet.

Figure 1.20(a) shows two end systems, a server and a client, connected to a computer network.

Consider the throughput for a file transfer from the server to the client.

The server is connected to the network with an access link of rate R_s and the client is connected to the network with an access link of rate R_c .

Now suppose that all the links in the core of the communication network have very high transmission rates, much higher than R_s and R_c .

Indeed, today, the core of the Internet is over-provisioned with high speed links that experience little congestion.

Also suppose that the only bits being sent in the entire network are those from the server to the client. Because the core of the computer network is like a wide pipe in this example, the rate at which bits can flow from source to destination is again the minimum of R_s and R_c , that is, throughput = $\min\{R_s, R_c\}$.

Therefore, the constraining factor for throughput in today's Internet is typically the access network.

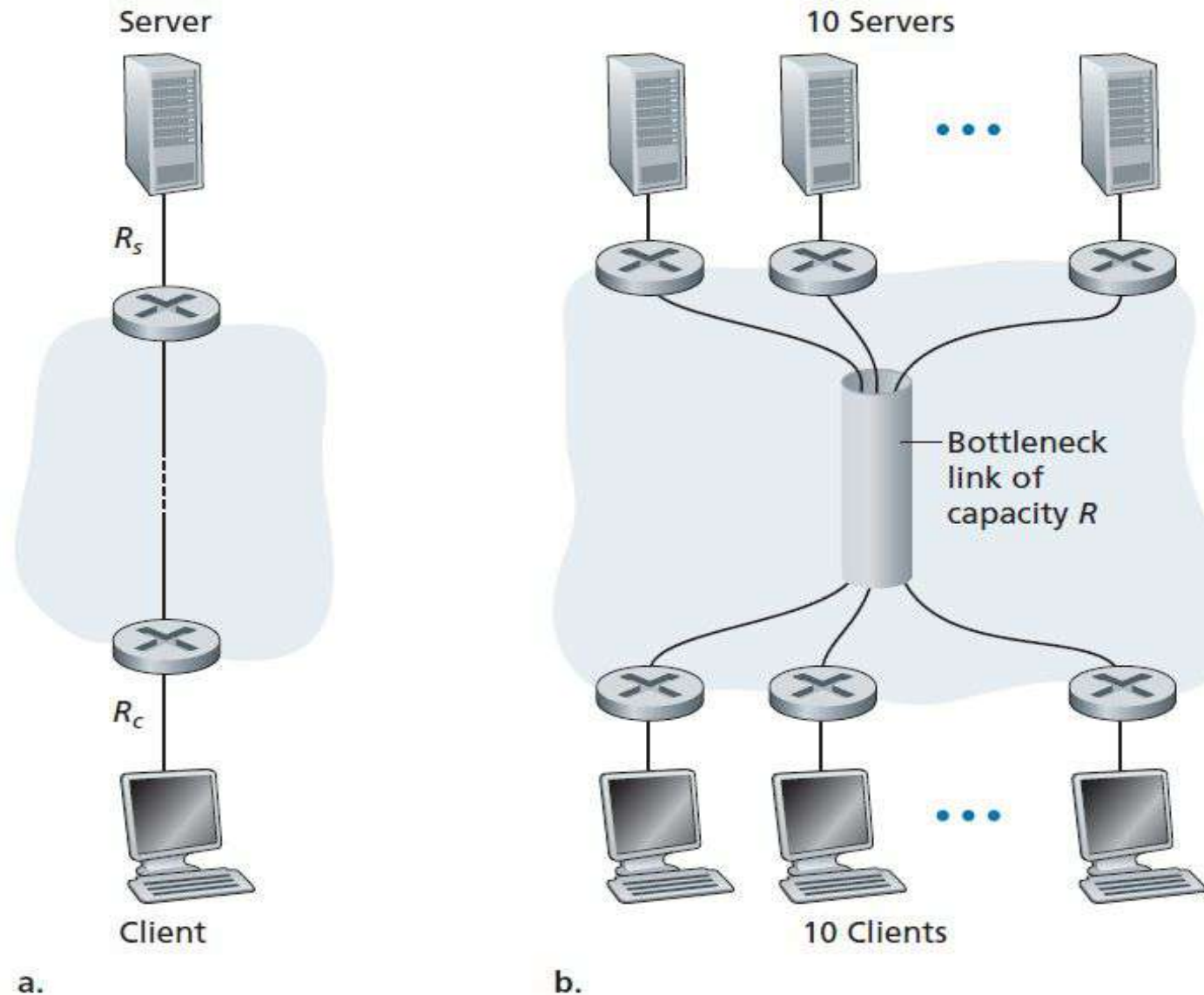


Figure 1.20 ♦ End-to-end throughput: (a) Client downloads a file from server; (b) 10 clients downloading with 10 servers

For a final example, consider Figure 1.20(b) in which there are 10 servers and 10 clients connected to the core of the computer network.

In this example, there are 10 simultaneous downloads taking place, involving 10 client-server pairs.

Suppose that these 10 downloads are the only traffic in the network at the current time.

As shown in the figure, there is a link in the core that is traversed by all 10 downloads.

Denote R for the transmission rate of this link R .

Let's suppose that all server access links have the same rate R_s , all client access links have the same rate R_c , and the transmission rates of all the links in the core—except the one common link of rate R —are much larger than R_s , R_c , and R .

Now we ask, what are the throughputs of the downloads? Clearly, if the rate of the common link, R , is large—say a hundred times larger than both R_s and R_c —then the throughput for each download will once again be $\min\{R_s, R_c\}$.

But what if the rate of the common link is of the same order as R_s and R_c ? What will the throughput be in this case? Let's take a look at a specific example.

Suppose $R_s = 2$ Mbps, $R_c = 1$ Mbps, $R = 5$ Mbps, and the common link divides its transmission rate equally among the 10 downloads.

Then the bottleneck for each download is no longer in the access network, but is now instead the shared link in the core, which only provides each download with 500 kbps of throughput.

Thus, the end-to-end throughput for each download is now reduced to 500 kbps.

The examples in Figure 1.19 and Figure 1.20(a) show that throughput depends on the transmission rates of the links over which the data flows.

We saw that when there is no other intervening traffic, the throughput can simply be approximated as the minimum transmission rate along the path between source and destination.

The example in Figure 1.20(b) shows that more generally the throughput depends not only on the transmission rates of the links along the path, but also on the intervening traffic.

In particular, a link with a high transmission rate may nonetheless be the bottleneck link for a file transfer if many other data flows are also passing through that link.

1.5 Protocol Layers and Their Service Models

1.5.1 Layered Architecture

Protocol Layering

Let's now turn our attention to network protocols.

To provide structure to the design of network protocols, network designers organize protocols—and the network hardware and software that implement the protocols—in **layers**.

Each protocol belongs to one of the layers,

We are again interested in the **services** that a layer offers to the layer above—the so-called **service model** of a layer.

each layer provides its service by

- (1) performing certain actions within that layer and by
- (2) using the services of the layer directly below it.

A protocol layer can be implemented in software, in hardware, or in a combination of the two.

Application-layer protocols—such as HTTP and SMTP—are almost always implemented in software in the end systems; so are transport-layer protocols.

Because the physical layer and data link layers are typically implemented in a network interface card (for example, Ethernet or WiFi interface cards) associated with a given link.

The network layer is often a mixed implementation of hardware and software.

a layer n protocol *distributed* among the end systems, packet switches, and other components that make up the network.

We mention, however, that some researchers and networking engineers are vehemently opposed to layering [Wakeman 1992].

One potential drawback of layering is that one layer may duplicate lower-layer functionality.

A second potential drawback is that functionality at one layer may need information that is present only in another layer; this violates the goal of separation of layers.

When taken together, the protocols of the various layers are called the **protocol stack**.

The Internet protocol stack consists of five layers: the physical, link, network, transport, and application layers, as shown in Figure 1.23.

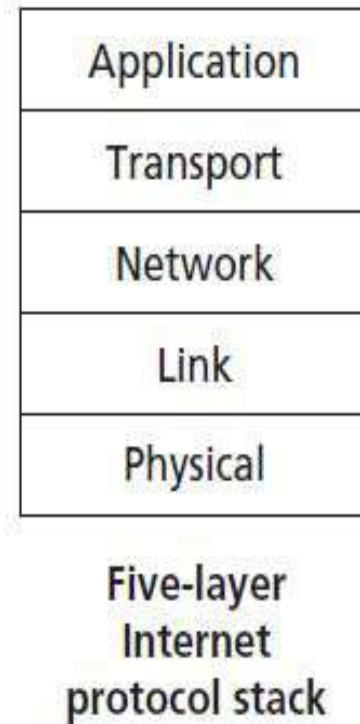


Figure 1.23 ♦ The Internet protocol stack

Application Layer

The application layer is where network applications and their application-layer protocols reside.

The Internet's application layer includes many protocols, such as the HTTP protocol (which provides for Web document request and transfer), SMTP (which provides for the transfer of e-mail messages), and FTP (which provides for the transfer of files between two end systems).

We'll see that certain network functions, such as the translation of human-friendly names for Internet end systems like `www.ietf.org` to a 32-bit network address, are also done with the help of a specific application-layer protocol, namely, the domain name system (DNS).

An application-layer protocol is distributed over multiple end systems, with the application in one end system using the protocol to exchange packets of information with the application in another end system.

We'll refer to this packet of information at the application layer as a **message**.

Transport Layer

The Internet's transport layer transports application-layer messages between application endpoints.

In the Internet, there are two transport protocols, TCP and UDP, either of which can transport application-layer messages.

TCP provides a connection-oriented service to its applications.

This service includes guaranteed delivery of application-layer messages to the destination and flow control (that is, sender/receiver speed matching).

TCP also breaks long messages into shorter segments and provides a congestion-control mechanism, so that a source throttles its transmission rate when the network is congested.

The UDP protocol provides a connectionless service to its applications.

This is a no-frills service that provides no reliability, no flow control, and no congestion control.

In this book, we'll refer to a transport-layer packet as a **segment**.

Network Layer

The Internet's network layer is responsible for moving network-layer packets known as **datagrams** from one host to another.

The Internet transport-layer protocol (TCP or UDP) in a source host passes a transport-layer segment and a destination address to the network layer

The network layer then provides the service of delivering the segment to the transport layer in the destination host.

The Internet's network layer includes the celebrated IP protocol, which defines the fields in the datagram as well as how the end systems and routers act on these fields.

There is only one IP protocol, and all Internet components that have a network layer must run the IP protocol.

The Internet's network layer also contains routing protocols that determine the routes that datagrams take between sources and destinations.

The Internet has many routing protocols.

Although the network layer contains both the IP protocol and numerous routing protocols, it is often simply referred to as the IP layer, reflecting the fact that IP is the glue that binds the Internet together.

Link Layer

To move a packet from one node (host or router) to the next node in the route, the network layer relies on the services of the link layer.

In particular, at each node, the network layer passes the datagram down to the link layer, which delivers the datagram to the next node along the route.

At this next node, the link layer passes the datagram up to the network layer.

The services provided by the link layer depend on the specific link-layer protocol that is employed over the link.

For example, some link-layer protocols provide reliable delivery, from transmitting node, over one link, to receiving node.

Note that this reliable delivery service is different from the reliable delivery service of TCP, which provides reliable delivery from one end system to another.

Examples of link-layer protocols include Ethernet, WiFi, and the cable access network's DOCSIS protocol.

As datagrams typically need to traverse several links to travel from source to destination, a datagram may be handled by different link-layer protocols at different links along its route. For example, a datagram may be handled by Ethernet on one link and by PPP on the next link. The network layer will receive a different service from each of the different link-layer protocols.

In this book, we'll refer to the link-layer packets as **frames**.

Physical Layer

While the job of the link layer is to move entire frames from one network element to an adjacent network element, the job of the physical layer is to move the *individual bits* within the frame from one node to the next.

The protocols in this layer are again link dependent and further depend on the actual transmission medium of the link (for example, twisted-pair copper wire, single-mode fiber optics).

For example, Ethernet has many physical-layer protocols: one for twisted-pair copper wire, another for coaxial cable, another for fiber, and so on.

In each case, a bit is moved across the link in a different way.

1.5.2 Encapsulation

Figure 1.24 shows the physical path that data takes down a sending end system's protocol stack, up and down the protocol stacks of an intervening link-layer switch and router, and then up the protocol stack at the receiving end system.

As we discuss later in this book, routers and link-layer switches are both packet switches. Similar to end systems, routers and link-layer switches organize their networking hardware and software into layers.

But routers and link-layer switches do not implement *all* of the layers in the protocol stack; they typically implement only the bottom layers.

As shown in Figure 1.24, link-layer switches implement layers 1 and 2; routers implement layers 1 through 3.

This means, for example, that Internet routers are capable of implementing the IP protocol (a layer 3 protocol), while link-layer switches are not.

We'll see later that while link-layer switches do not recognize IP addresses, they are capable of recognizing layer 2 addresses, such as Ethernet addresses.

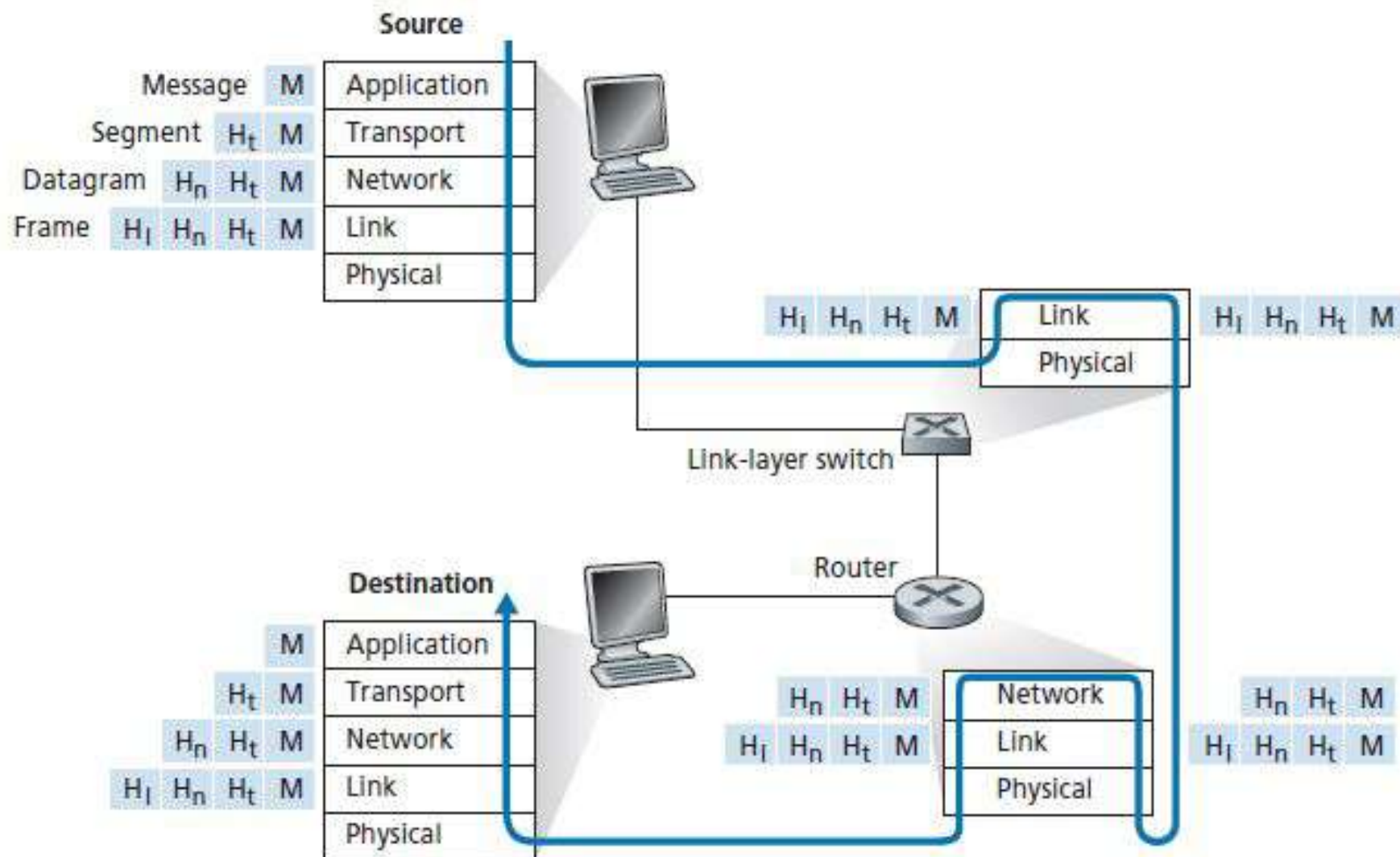


Figure 1.24 ♦ Hosts, routers, and link-layer switches; each contains a different set of layers, reflecting their differences in functionality

Note that hosts implement all five layers; this is consistent with the view that the Internet architecture puts much of its complexity at the edges of the network.

Figure 1.24 also illustrates the important concept of **encapsulation**.

At the sending host, an **application-layer message** (M in Figure 1.24) is passed to the transport layer.

In the simplest case, the transport layer takes the message and appends additional information (so-called transport-layer header information, H, in Figure 1.24) that will be used by the receiver-side transport layer.

The application-layer message and the transport-layer header information together constitute the **transport-layer segment**.

The transport-layer segment thus encapsulates the application-layer message.

The added information might include information allowing the receiver-side transport layer to deliver the message up to the appropriate application, and error-detection bits that allow the receiver to determine whether bits in the message have been changed in route.

The transport layer then passes the segment to the network layer, which adds network-layer header information (H_n in Figure 1.24) such as source and destination end system addresses, creating a **network-layer datagram**.

The datagram is then passed to the link layer, which (of course!) will add its own link-layer header information and create a **link-layer frame**.

Thus, we see that at each layer, a packet has two types of fields: header fields and a **payload field**. The payload is typically a packet from the layer above.

The process of encapsulation can be more complex than that described above.

1.6 Networks Under Attack

The field of network security is about how the bad guys can attack computer networks and about how we can defend networks against those attacks, or better yet, design new architectures that are immune to such attacks in the first place.

Given the frequency and variety of existing attacks as well as the threat of new and more destructive future attacks, network security has become a central topic in the field of computer networking.

The Bad Guys Can Put Malware into Your Host Via the Internet

We attach devices to the Internet because we want to receive/send data from/to the Internet. This includes all kinds of good stuff, including Instagram posts, Internet search results, streaming music, video conference calls, streaming movies, and so on.

But, unfortunately, along with all that good stuff comes malicious stuff—collectively known as **malware**—that can also enter and infect our devices.

Once malware infects our device it can do all kinds of devious things, including deleting our files and installing spyware that collects our private information, such as social security numbers, passwords, and keystrokes, and then sends this (over the Internet, of course!) back to the bad guys.

Our compromised host may also be enrolled in a network of thousands of similarly compromised devices, collectively known as a **botnet**, which the bad guys control and leverage for spam e-mail distribution or distributed denial-of-service attacks (soon to be discussed) against targeted hosts.

Much of the malware out there today is **self-replicating**: once it infects one host, from that host it seeks entry into other hosts over the Internet, and from the newly infected hosts, it seeks entry into yet more hosts.

In this manner, self- replicating malware can spread exponentially fast.

The Bad Guys Can Attack Servers and Network Infrastructure

Another broad class of security threats are known as **denial-of-service (DoS) attacks**.

As the name suggests, a DoS attack renders a network, host, or other piece of infrastructure unusable by legitimate users.

Web servers, e-mail servers, DNS servers (discussed in Chapter 2), and institutional networks can all be subject to DoS attacks.

Most Internet DoS attacks fall into one of three categories:

- *Vulnerability attack.*
- *Bandwidth flooding.*
- *Connection flooding.*

Let's now explore the bandwidth-flooding attack in more detail.

it's evident that if the server has an access rate of R bps, then the attacker will need to send traffic at a rate of approximately R bps to cause damage.

If R is very large, a single attack source may not be able to generate enough traffic to harm the server.

In a **distributed DoS (DDoS)** attack, illustrated in Figure 1.25, the attacker controls multiple sources and has each source blast traffic at the target.

With this approach, the aggregate traffic rate across all the controlled sources needs to be approximately R to cripple the service.

DDoS attacks leveraging botnets with thousands of comprised hosts are a common occurrence today [DAM 2020].

DDoS attacks are much harder to detect and defend against than a DoS attack from a single host.

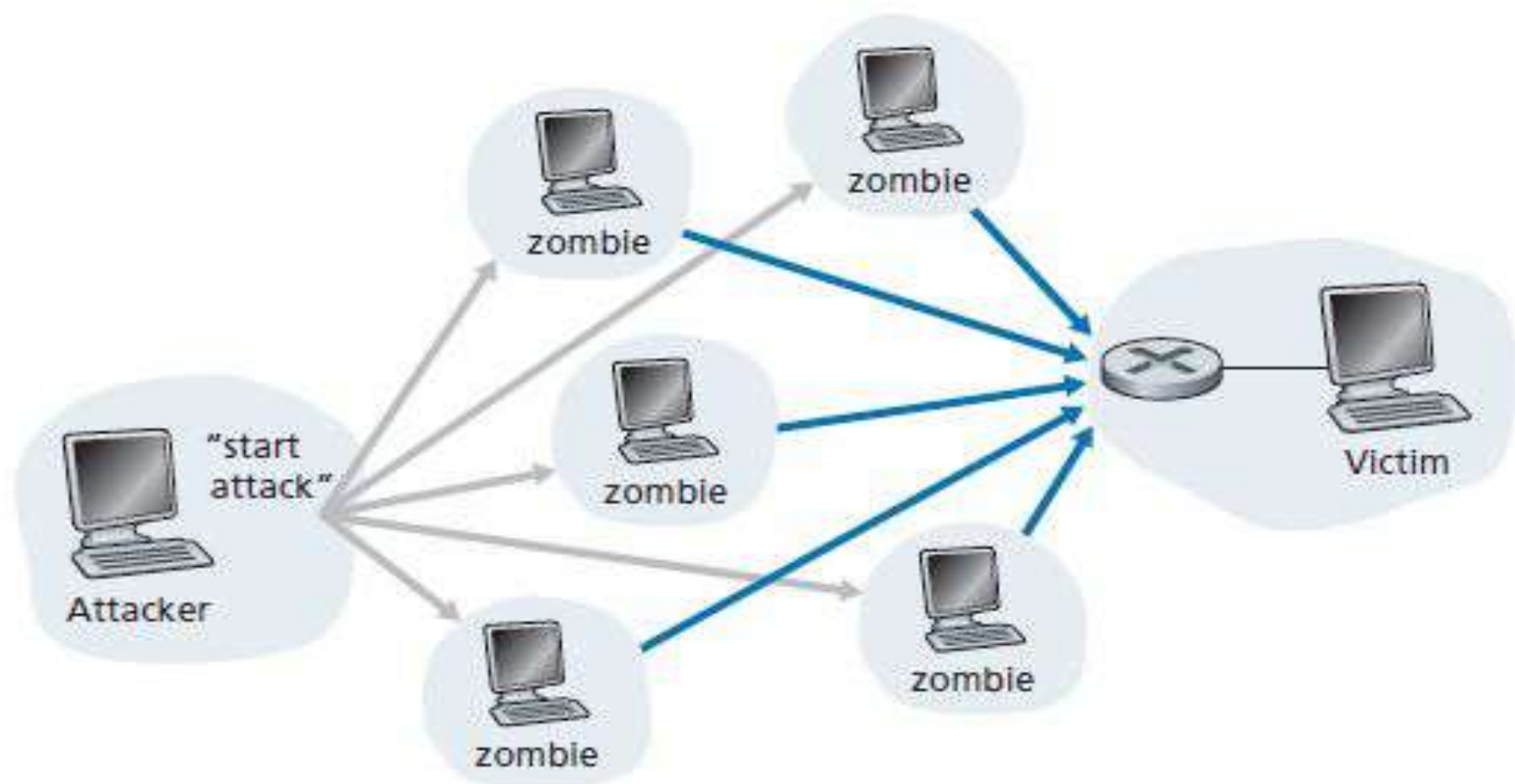


Figure 1.25 ♦ A distributed denial-of-service attack

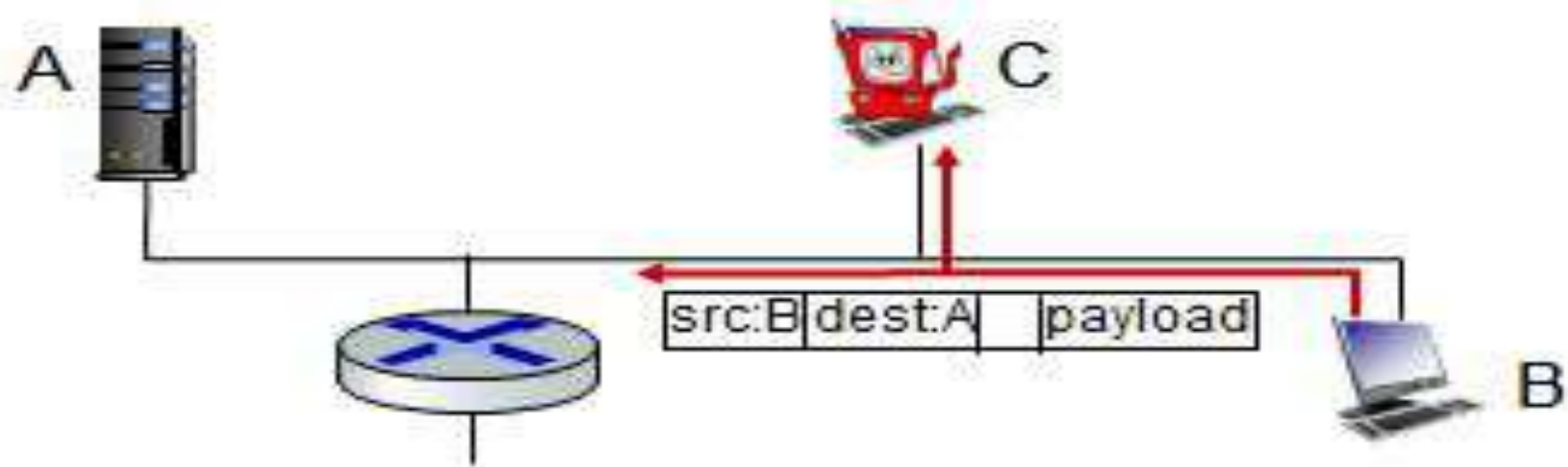
The Bad Guys Can Sniff Packets

Many users today access the Internet via wireless devices, such as WiFi-connected laptops or handheld devices with cellular Internet connections

placing a passive receiver in the vicinity of the wireless transmitter, that receiver can obtain a copy of every packet that is transmitted! These packets can contain all kinds of sensitive information, including passwords, social security numbers, trade secrets, and private personal messages.

A passive receiver that records a copy of every packet that flies by is called a **packet sniffer**.

Sniffers can be deployed in wired environments as well



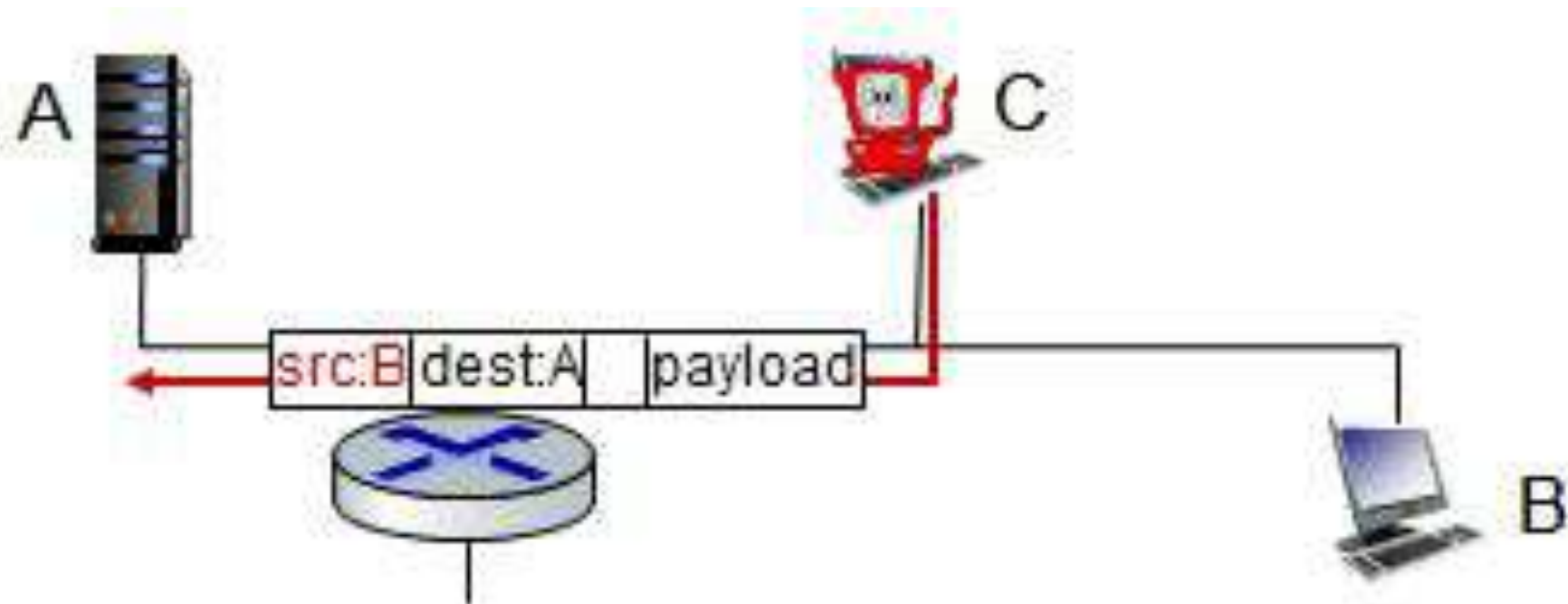
Sniffed packets can then be analyzed offline for sensitive information.

Packet-sniffing software is freely available at various Web sites and as commercial products.

Because packet sniffers are passive—that is, they do not inject packets into the channel—they are difficult to detect.

So, when we send packets into a wireless channel, we must accept the possibility that some bad guy may be recording copies of our packets.

As you may have guessed, some of the best defenses against packet sniffing involve cryptography.



The Bad Guys Can Masquerade as Someone You Trust

It is surprisingly easy to create a packet with an arbitrary source address, packet content, and destination address and then transmit this hand-crafted packet into the Internet, which will dutifully forward the packet to its destination.

Imagine the unsuspecting receiver (say an Internet router) who receives such a packet, takes the (false) source address as being truthful, and then performs some command embedded in the packet's contents (say modifies its forwarding table).

The ability to inject packets into the Internet with a false source address is known as **IP spoofing**, and is but one of many ways in which one user can masquerade as another user.

To solve this problem, we will need *end-point authentication*, that is, a mechanism that will allow us to determine with certainty if a message originates from where we think it does.

In closing this section, it's worth considering how the Internet got to be such an insecure place in the first place.

The answer, in essence, is that the Internet was originally designed to be that way, based on the model of “a group of mutually trusting users attached to a transparent network” [Blumenthal 2001]—a model in which (by definition) there is no need for security.

Many aspects of the original Internet architecture deeply reflect this notion of mutual trust.

But today's Internet certainly does not involve “mutually trusting users.”

Nonetheless, today's users still need to communicate when they don't necessarily trust each other, may wish to communicate anonymously, may communicate indirectly through third parties (e.g., Web caches, which we'll study in Chapter 2, or mobility-assisting agents, which we'll study in Chapter 7), and may distrust the hardware, software, and even the air through which they communicate.

We should keep in mind that communication among mutually trusted users is the exception rather than the rule.

1.7 History of Computer Networking and the Internet

1.7.1 The Development of Packet Switching: 1961–1972

The field of computer networking and today's Internet trace their beginnings back to the early 1960s, when the telephone network was the world's dominant communication network.

Given the increasing importance of computers in the early 1960s and the advent of timeshared computers, it was perhaps natural to consider how to hook computers together so that they could be shared among geographically distributed users.

Three research groups around the world, each unaware of the others' work [Leiner 1998], began inventing packet switching as an efficient and robust alternative to circuit switching.

The work at MIT, Rand, and the NPL laid the foundations for today's Internet.

an overall plan for the ARPAnet [Roberts 1967], the first packet-switched computer network and a direct ancestor of today's public Internet.

By 1972, ARPAnet had grown to approximately 15 nodes

The first host-to-host protocol between ARPAnet end systems, known as the network-control protocol (NCP),

the first e-mail program in 1972.

1.7.2 Proprietary Networks and Internetworking:1972–1980

The number of networks was growing.

The three key Internet protocols that we see today—TCP, UDP, and IP—were conceptually in place by the end of the 1970s.

1.7.3 A Proliferation of Networks: 1980–1990

By the end of the 1970s, approximately two hundred hosts were connected to the ARPAnet. By the end of the 1980s the number of hosts connected to the public Internet, a confederation of networks looking much like today's Internet, would reach a hundred thousand.

January 1, 1983 saw the official deployment of TCP/IP as the new standard host protocol for ARPAnet (replacing the NCP protocol).

The transition [RFC 801] from NCP to TCP/IP was a flag day event—all hosts were required to transfer over to TCP/IP as of that day.

The DNS, used to map between a human-readable Internet name (for example, `gaia.cs.umass.edu`) and its 32-bit IP address, was also developed [RFC 1034].

Paralleling this development of the ARPAnet (which was for the most part a US effort), in the early 1980s the French launched the Minitel project, an ambitious plan to bring data networking into everyone's home.

The Minitel was in a large proportion of French homes 10 years before most Americans had ever heard of the Internet.

1.7.4 The Internet Explosion: The 1990s

The main event of the 1990s was to be the emergence of the World Wide Web application, which brought the Internet into the homes and businesses of millions of people worldwide.

Berners-Lee and his associates developed initial versions of HTML, HTTP, a Web server, and a browser—the four key components of the Web.

Around the end of 1993 there were about two hundred Web servers in operation

At about this time several researchers were developing Web browsers with GUI interfaces

By 1995, university students were using Netscape browsers to surf the Web on a daily basis. At about this time companies—big and small—began to operate Web servers and transact commerce over the Web.

In 1996, Microsoft started to make browsers, which started the browser war between Netscape and Microsoft, which Microsoft won a few years later [Cusumano 1998].

The second half of the 1990s was a period of tremendous growth and innovation for the Internet, with major corporations and thousands of startups creating Internet products and services.

By the end of the millennium the Internet was supporting hundreds of popular applications, including four killer applications:

- E-mail, including attachments and Web-accessible e-mail
- The Web, including Web browsing and Internet commerce
- Instant messaging, with contact lists
- Peer-to-peer file sharing of MP3s, pioneered by Napster

a number of companies emerged as big winners in the Internet space, including Microsoft, Cisco, Yahoo, eBay, Google, and Amazon.

1.7.5 The New Millennium

- Since the beginning of the millennium, we have been seeing aggressive deployment of broadband Internet access to homes—not only cable modems and DSL but also fiber to the home, and now 5G fixed wireless

This high-speed Internet access has set the stage for a wealth of video applications, including the distribution of user-generated video (for example, YouTube), on-demand streaming of movies and television shows (e.g., Netflix), and multi-person video conference (e.g., Skype, Facetime, and Google Hangouts).

- The increasing ubiquity of high-speed wireless Internet access is not only making it possible to remain constantly connected while on the move,

The number of wireless devices connecting to the Internet surpassed the number of wired devices in 2011.

This high-speed wireless access has set the stage for the rapid emergence of hand-held computers (iPhones, Androids, iPads, and so on), which enjoy constant and untethered access to the Internet.

- Online social networks—such as Facebook, Instagram, Twitter, and WeChat (hugely popular in China)—have created massive people networks on top of the Internet. Many of these social networks are extensively used for messaging as well as photo sharing. Many Internet users today “live” primarily within one or more social networks.

online service providers, such as Google and Microsoft, have deployed their own extensive private networks, which not only connect together their globally distributed data centers, but are used to bypass the Internet as much as possible by peering directly with lower-tier ISPs. As a result, Google provides search results and e-mail access almost instantaneously, as if their data centers were running within one’s own computer.

- Many Internet commerce companies are now running their applications in the “cloud”—such as in Amazon’s EC2, in Microsoft’s Azure, or in the Alibaba Cloud.

1.8 Summary

Road-Mapping This Book

Our road map is the sequence of chapters of this book:

1. Computer Networks and the Internet
2. Application Layer
3. Transport Layer
4. Network Layer: Data Plane
5. Network Layer: Control Plane
6. The Link Layer and LANs
7. Wireless and Mobile Networks
8. Security in Computer Networks

Chapters 2 through 6 are the five core chapters of this book.

You should notice that these chapters are organized around the top four layers of the five-layer Internet protocol.

The second half of the book—Chapters 7 and 8—zooms in on two enormously important (and somewhat independent) topics in modern computer networking.

In Chapter 7, we examine wireless and mobile networks, including wireless LANs (including WiFi and Bluetooth), Cellular networks (including 4G and 5G), and mobility.

Chapter 8, which addresses security in computer networks, first looks at the underpinnings of encryption and network security, and then we examine how the basic theory is being applied in a broad range of Internet contexts.